

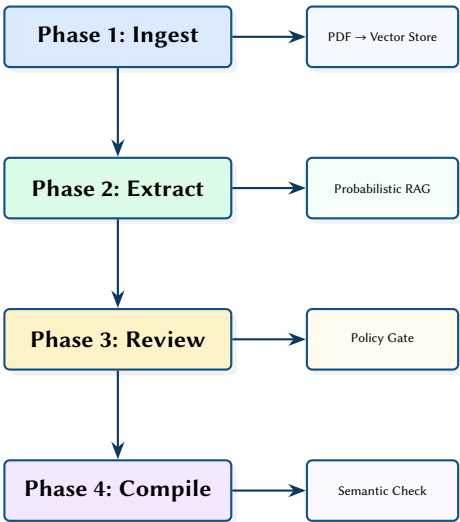
LLM-Powered Network Configuration Parsing and Cross-Vendor Compilation

RAG-Based Extraction, Human-in-the-Loop Review, and Semantic Validation

Simon Knight

March 2026 • Version 1.0.0

Last updated: March 18, 2026



Abstract. Network configuration extraction is fundamentally a probabilistic graph construction problem over uncertain vendor syntax. This report presents a framework that models network topology as a typed property graph where every node and edge carries a statistical confidence score and strict evidence provenance. By treating link inference as a probabilistic join over weak syntax signals, the system formalises ambiguity, routing uncertain extractions to an evidence-enforced human review gate. Once verified, the intent graph compiles to deterministic vendor outputs (Cisco IOS, Arista EOS) validated against deep semantic constraints. The pipeline is implemented using Retrieval-Augmented Generation over vendor documentation to condition the probabilistic extraction.

Contents

1	Introduction and Motivation	2
2	Foundational Data Models and Mathematics	2
2.1	The Graph Intermediate Representation (IR)	2
2.2	Semantic Backpropagation and Iterative Refinement	4
2.3	Confidence Routing and Probabilistic Joins	5
2.4	Formalism of Semantic Invariants	6
3	Operational Case Study: Resolving Ambiguous OSPF Intent	7
3.1	Extraction Generation and Review Flow	7
3.2	Iterative Intent Resolution Algorithm	9
3.3	Semantic Validation Harness	10
4	System Architecture and Implementation	11
4.1	Knowledge Base Ingestion (Phase 1)	11
4.2	Hybrid GraphRAG: Linking Vectors to Entities	12
4.3	Extraction Architecture and Structured Outputs (Phase 2)	13
4.4	Review Persistence (Phase 3)	14
4.5	Compilation (Phase 4)	15
5	Performance and Evaluation Metrics	16
5.1	Limitations and Trade-offs	16
6	Conclusion and Future Work	17
6.1	Future Work	17
	Revision History	19
A	Intermediate Representation JSON Schema	19
LLM	Large Language Model	13
RAG	Retrieval-Augmented Generation	13
IR	Intermediate Representation	2
HITL	Human-in-the-Loop	8
API	Application Programming Interface	16
OSPF	Open Shortest Path First	2
BGP	Border Gateway Protocol	2

1 Introduction and Motivation

The abstraction of network intent is plagued by the ambiguity of vendor-specific syntax. The same logical intent (“interface Ethernet1 has IP 10.0.0.1/30 in VLAN 100”) is expressed idiosyncratically across Cisco IOS, Arista EOS, and Juniper JunOS. Traditional parsers attempt to solve this via hand-written regular expressions or TextFSM templates, mapping syntax directly to schema. This approach is fundamentally brittle: it fails when syntax evolves, and it forces engineers to maintain an ever-growing library of deterministic rules for inherently non-deterministic inputs.

This framework introduces a mathematical shift in perspective. Instead of treating parsing as a deterministic regex problem, we model extraction as a probabilistic graph construction problem over uncertain evidence.

By treating the source configuration as a set of weak signals rather than absolute truths, we construct a vendor-neutral Intermediate Representation (IR) where every extracted field carries a calculable confidence score. This transforms opaque parsing failures into measurable uncertainty, which can be explicitly routed to human review or automatically rejected by policy gates.

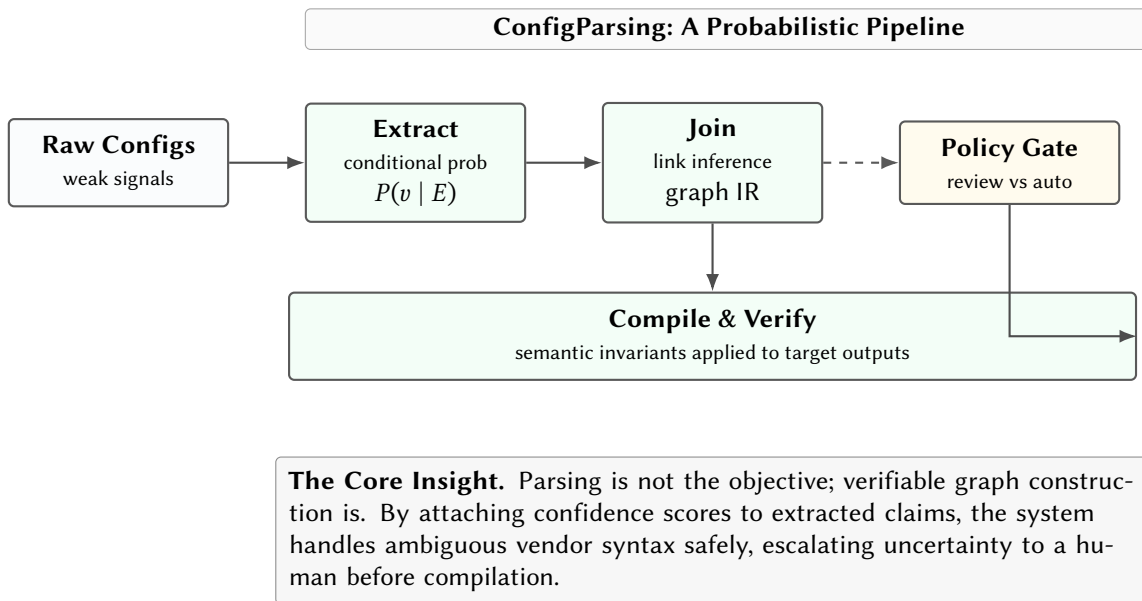


Figure 1. From raw configurations to a validated graph IR, highlighting the probabilistic approach to extraction.

: Evidence-Backed Extraction

Every extracted field must carry a probability score $P(v) \in [0.0, 1.0]$ and strict evidence citations linking the value to source configuration lines and reference documentation. Uncertainty dictates operational routing.

2 Foundational Data Models and Mathematics

Before discussing the operational pipeline, we define the foundational structures. Parsing is not the objective; the objective is to build a verifiable, typed property graph where every edge carries statistical confidence.

2.1 The Graph Intermediate Representation (IR)

The output is a vendor-neutral IR modelling:

- **Interfaces** — name, IP addresses, VLANs, status
- **Routing** — Open Shortest Path First (OSPF) areas, Border Gateway Protocol (BGP) peers, static routes
- **Services** — DHCP, NTP, DNS, ACLs

Table 1. Multi-Vendor Intent Mapping Matrix: Unified IR properties vs. legacy syntax.

IR Property	Cisco IOS Syntax	Arista EOS Syntax	Juniper JunOS Syntax
iface.ip	ip address 1.1.1.1	ip address 1.1.1.1/24	family inet address
ospf.area	network x.x.x.x area	ip ospf area x.x.x.x	set protocols ospf area
bgp.peer	neighbor 1.1.1.1 remote-as	neighbor 1.1.1.1 remote-as	set protocols bgp group
vlan.id	vlan 100	vlan 100	set vlans vlan-id 100

Each extracted field includes:

```
@dataclass
class ExtractedField:
    value: Any
    confidence: float          # 0.0 - 1.0
    config_evidence: list[str] # Source config line numbers
    doc_evidence: list[str]    # ChromaDB chunk IDs
```

Core Data Model (Entity Relationship Diagram)

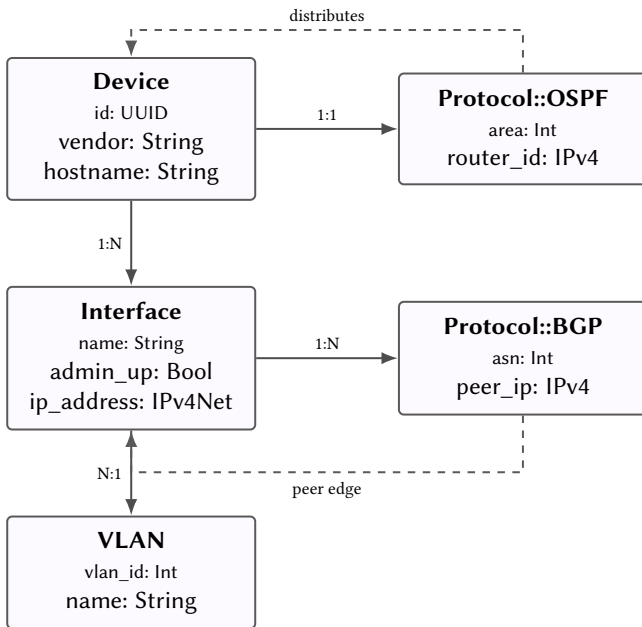


Figure 2. Entity Relationship Diagram (ERD) mapping out the strict schema of the Intermediate Representation with orthogonal routing.

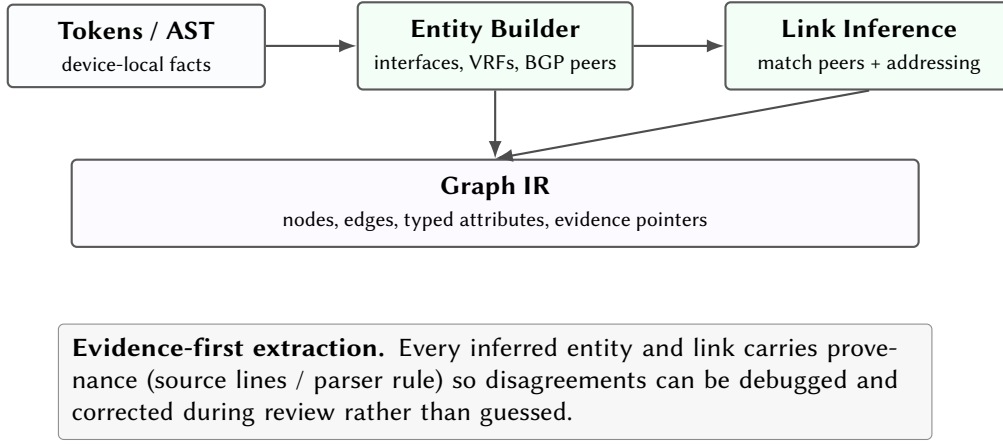


Figure 3. Topology extraction as a graph construction problem: facts are lifted into entities, then joined into links with explicit provenance.

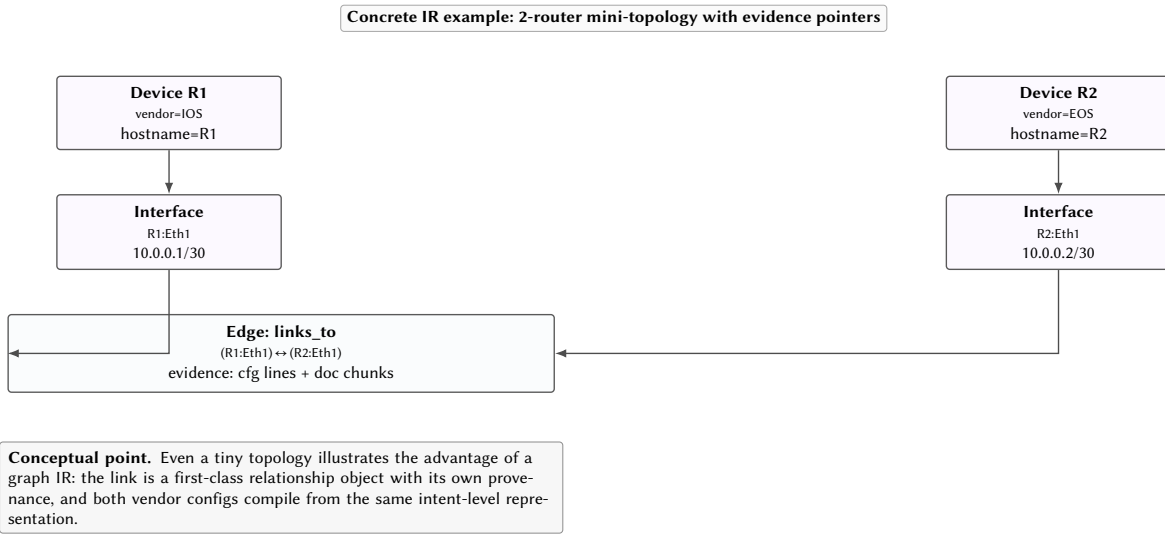


Figure 4. A minimal graph IR instance. Devices, interfaces, and a link edge are represented explicitly; evidence attaches at the edge/field level for auditability.

2.2 Semantic Backpropagation and Iterative Refinement

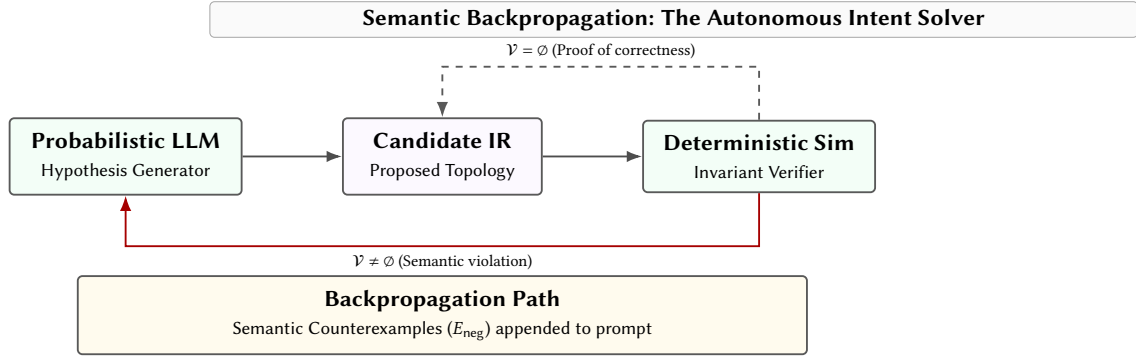
A critical innovation of this framework is the coupling of probabilistic extraction with deterministic verification via **Semantic Backpropagation**. We define the verification engine (Batfish) as a deterministic loss function $\mathcal{L}$ that maps a compiled configuration $\mathcal{C}$ to a set of semantic violations $\mathcal{V}$:

$$\mathcal{V} = \mathcal{L}(\mathcal{C}, \mathcal{I}_{\text{assert}}) \quad (1)$$

where $\mathcal{I}_{\text{assert}}$ represents the structural invariants defined in the IR. When $\mathcal{V} \neq \emptyset$, the resulting semantic counterexamples E_{neg} are backpropagated into the extraction engine. The probability of an extracted value v is then recalculated using the violations as negative conditioning:

$$c'_v = P(v \mid E_{\text{cfg}}, E_{\text{doc}}, E_{\text{neg}}) \quad (2)$$

This closed-loop system allows the framework to autonomously search the hypothesis space of possible network topologies, iteratively refining its extraction until the deterministic simulator proves that all semantic invariants are satisfied.



The PhD Insight. Most "AI" systems stop at generation. By using a deterministic simulator as a feedback signal, we convert an opaque generative model into an autonomous solver that can mathematically prove its own outputs satisfy network intent before a human ever sees them.

Figure 5. The closed-loop Semantic Backpropagation architecture. Deterministic simulator failures act as negative evidence (E_{neg}), driving iterative probabilistic refinement.

2.3 Confidence Routing and Probabilistic Joins

We define the extracted network topology as a set of random variables. For any extracted field v (e.g., an interface IP or a BGP ASN), we compute a confidence score c_v which models the probability of correctness conditioned on the available evidence:

$$c_v = P(v \mid E_{\text{cfg}}, E_{\text{doc}}) \quad (3)$$

where E_{cfg} represents the specific configuration token span and E_{doc} represents the retrieved documentation chunks.

Link inference between two device interfaces A and B is treated not as a deterministic string match, but as a probabilistic join. The probability of an edge $E_{A,B}$ is computed over multiple weak signals S_i (e.g., subnet overlap, LLDP hints, description strings):

$$P(E_{A,B}) = \sigma \left(\sum_i w_i \cdot \phi(S_i) \right) \quad (4)$$

This mathematical formalisation enables explicit operational control. We define two strict policy thresholds, t_{review} and t_{reject} , which route the flow of data based on uncertainty.

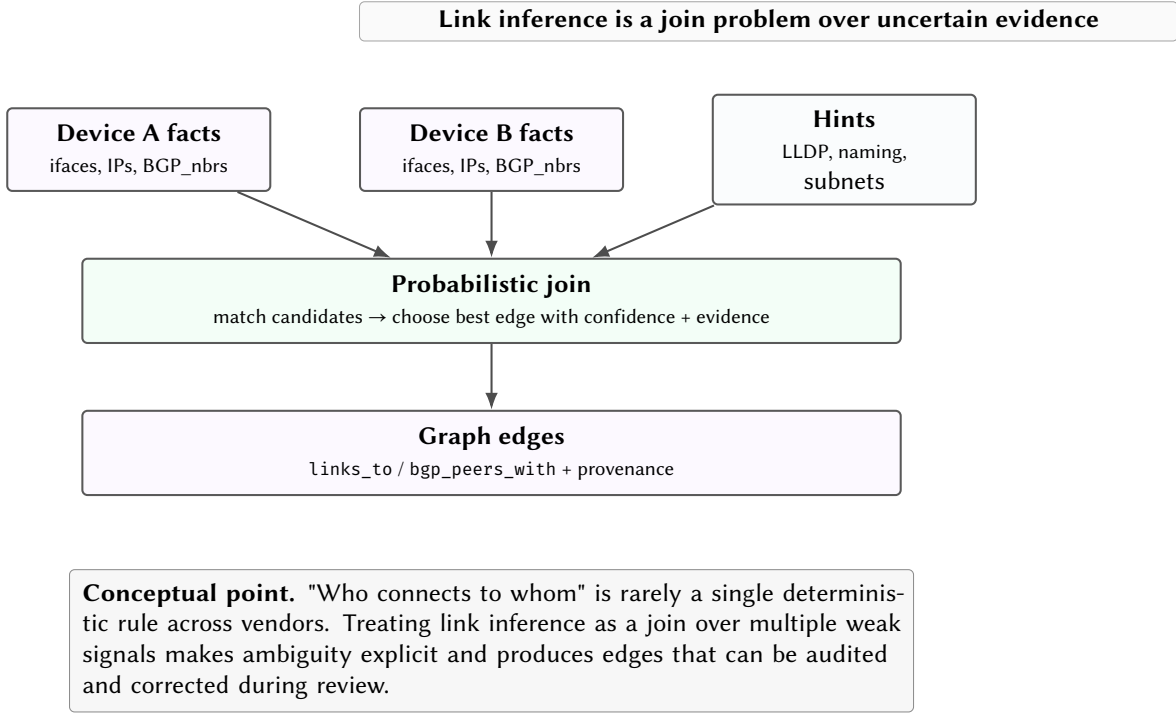


Figure 6. Link inference as a constrained join. Multiple evidence sources contribute to candidate matches; the chosen edge carries confidence and provenance.

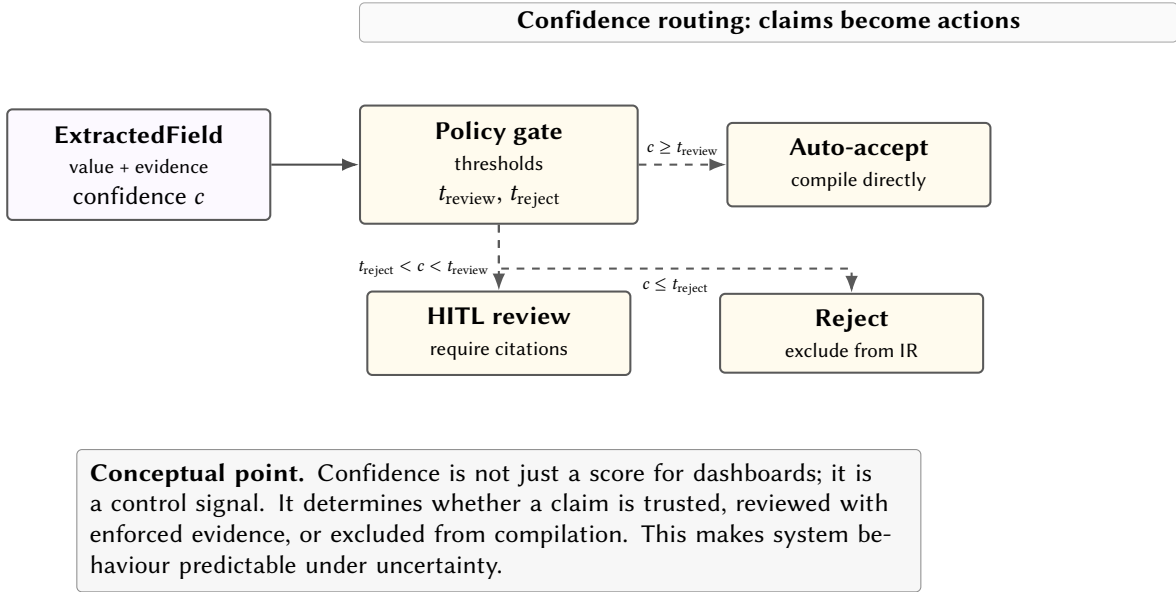


Figure 7. Confidence routing converts extraction uncertainty into operational behaviour: accept, review, or reject with explicit thresholds.

2.4 Formalism of Semantic Invariants

To move beyond syntactic "look-alike" parsing, we formally define network intent as a set of First-Order Logic (FOL) predicates over the graph [IR](#). A "correct" configuration is one whose compilation satisfies the entire set of predicates $\mathcal{P}$.

For example, the invariant of **Strong Reachability** between two nodes a and b is defined as:

$$\forall a, b \in \mathcal{N}, \text{Reach}(a, b) \iff \exists P = (e_1, e_2, \dots, e_n) : \text{Path}(a, b, P) \wedge \forall e \in P, \text{Status}(e) = \text{UP} \quad (5)$$

Similarly, **VLAN Isolation** between two subnets S_1 and S_2 is defined by the absence of a leak predicate:

$$\text{Isolate}(S_1, S_2) \iff \neg \exists x : (\text{Route}(S_1, x) \in \text{RIB}_{S_2}) \vee (\text{Route}(S_2, x) \in \text{RIB}_{S_1}) \quad (6)$$

By elevating intent to FOL predicates, the "Proof of Correctness" in the iterative refinement loop becomes a formal verification task. The system does not just guess if a config is right; it simulates the control plane to prove that these predicates hold true.

3 Operational Case Study: Resolving Ambiguous OSPF Intent

To demonstrate the mathematical framework in practice, consider a network migration from a legacy Cisco IOS router to a modern Arista EOS device. The source configuration contains an ambiguous OSPF definition where the network statement wildcard mask is implicitly defined, creating uncertainty about which interfaces are actually participating in the routing domain.

3.1 Extraction Generation and Review Flow

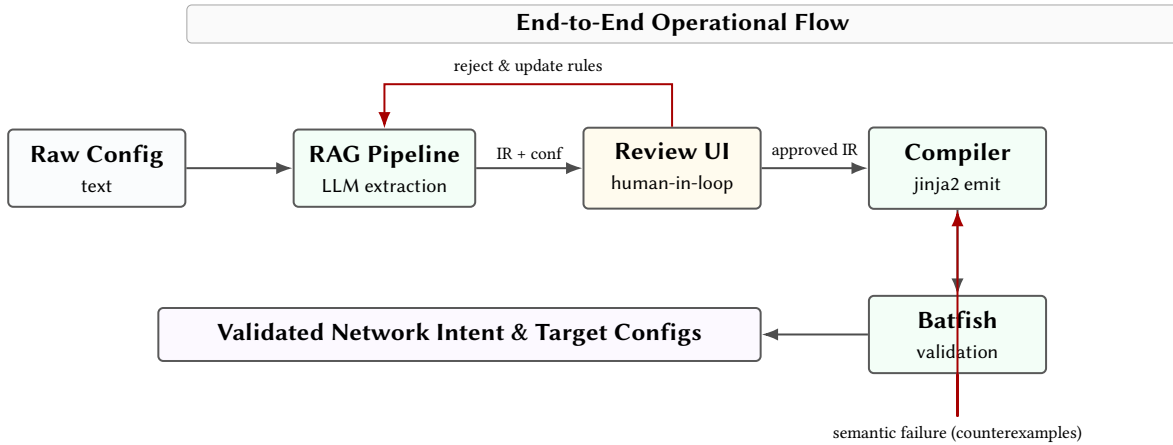


Figure 8. End-to-end sequence of data flow, showing extraction, HITL review, compilation, and semantic validation.

The extraction engine processes the raw Cisco IOS text, conditioning the probabilistic model on the retrieved vendor documentation for legacy IOS OSPF syntax. Because the wildcard mask overlaps multiple subnets unpredictably, the engine proposes an OSPF area assignment but assigns it a low confidence score ($c_v = 0.55$), falling below the automatic acceptance threshold ($t_{\text{review}} = 0.80$).

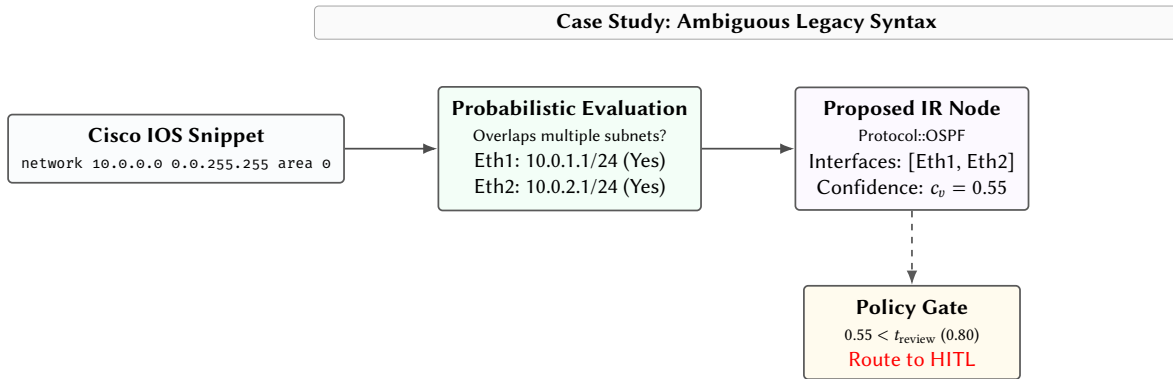


Figure 9. The probabilistic engine evaluates an ambiguous OSPF wildcard mask against active interfaces, resulting in a low-confidence IR node that triggers the policy gate.

1. Raw Cisco IOS

```

interface Ethernet1
ip addr 10.0.1.1 255.255.255.0
!
router ospf 1
network 10.0.0.0 0.0.255.255
  ↳ area 0

```

2. Extracted IR (JSON)

```

{
  "protocol": "ospf",
  "area": 0,
  "interfaces": ["Eth1", "Eth2"],
  "_meta": {
    "confidence": 0.55,
    "doc_ref": "ios:ospf:p4",
    "status": "flagged"
  }
}

```

3. Compiled Arista EOS

```

interface Ethernet1
ip address 10.0.1.1/24
ip ospf area 0.0.0.0
!
router ospf 1
router-id 10.0.1.1

```

Figure 10. The lifecycle of configuration intent: from ambiguous legacy syntax, to a quantifiable intermediate representation, to strict target emission.

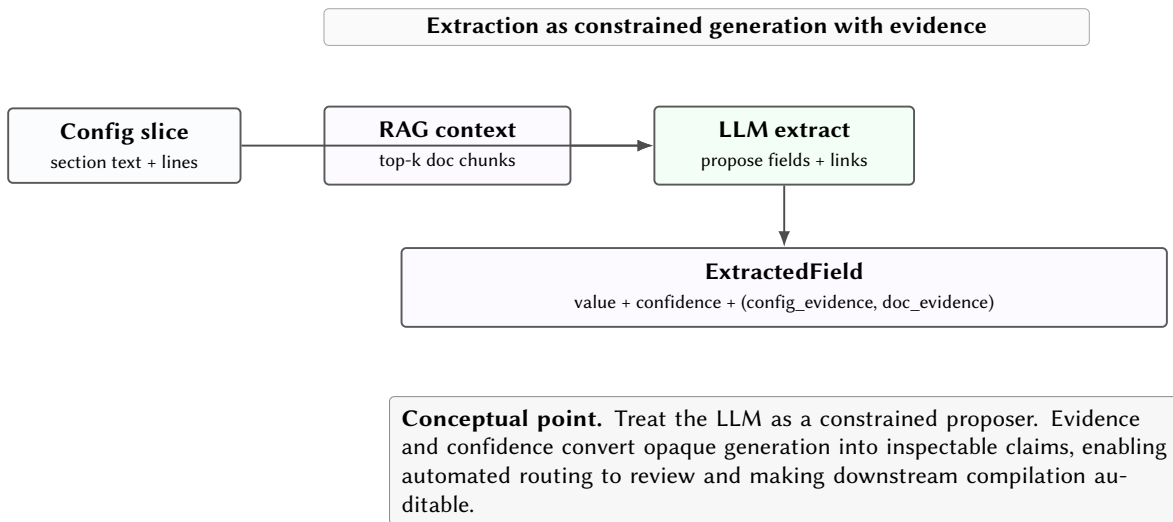


Figure 11. Evidence-backed extraction. Config text and retrieved docs jointly condition the LLM, which outputs values plus provenance, not just a best-guess parse.

Design Decision 1: Hallucination Protection

Fields extracted with confidence below the configurable threshold (default 0.6) are automatically flagged for Human-in-the-Loop (HITL) review. The evidence chain (config line → doc chunk → extracted value) enables reviewers to verify the LLM’s reasoning rather than trusting the output blindly.

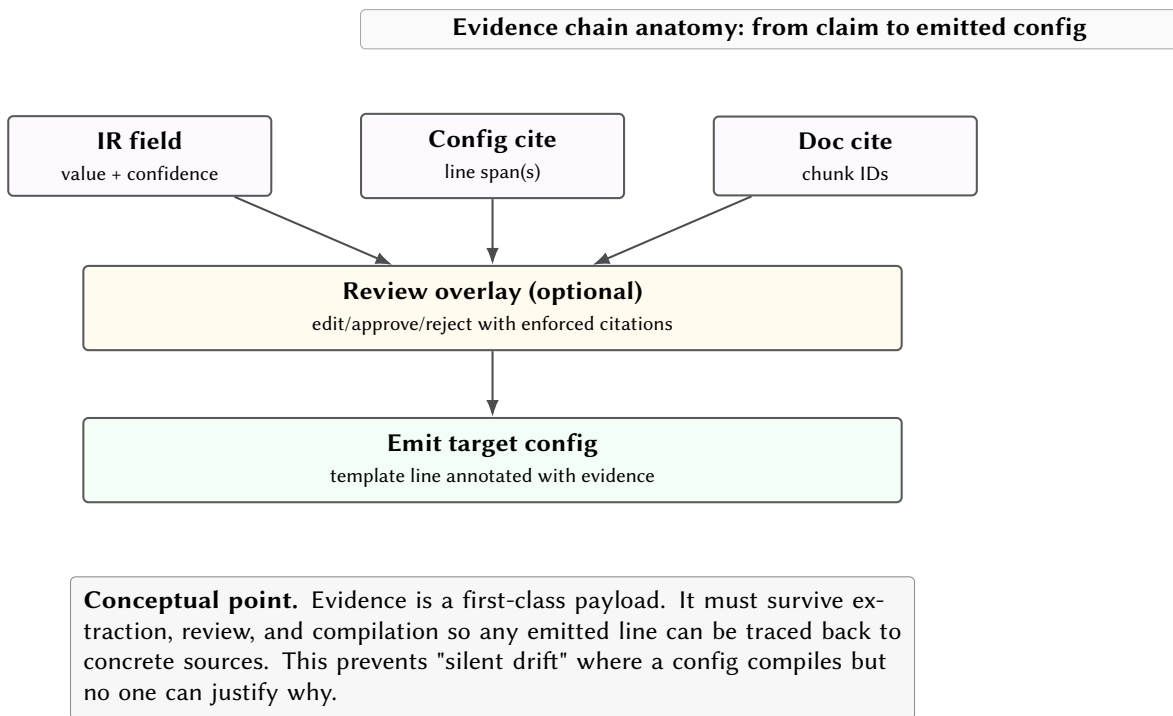


Figure 12. End-to-end evidence chain. The same provenance objects support auditing, review enforcement, and annotated compilation outputs.

To resolve this ambiguity, the uncertainty is automatically routed to a web-based review interface. The interface presents the flagged OSPF area extraction alongside its complete evidence chain. The reviewing engineer inspects the cited Cisco IOS documentation chunk and the raw config lines, confirming the proposed area is incorrect for the specific subnets.

The reviewer executes an **Edit** action, correcting the OSPF area assignment. Crucially, the system enforces that all manual edits must cite a concrete evidence source. The engineer cites the correct configuration line, setting the confidence score to 1.0. This human intervention is saved as a durable overlay, preventing the same ambiguity from blocking future runs.

- **Approve** — Accept the extraction as-is (confidence set to 1.0).
- **Edit** — Modify the value with mandatory evidence citation.
- **Reject** — Mark as incorrect; excluded from compilation.

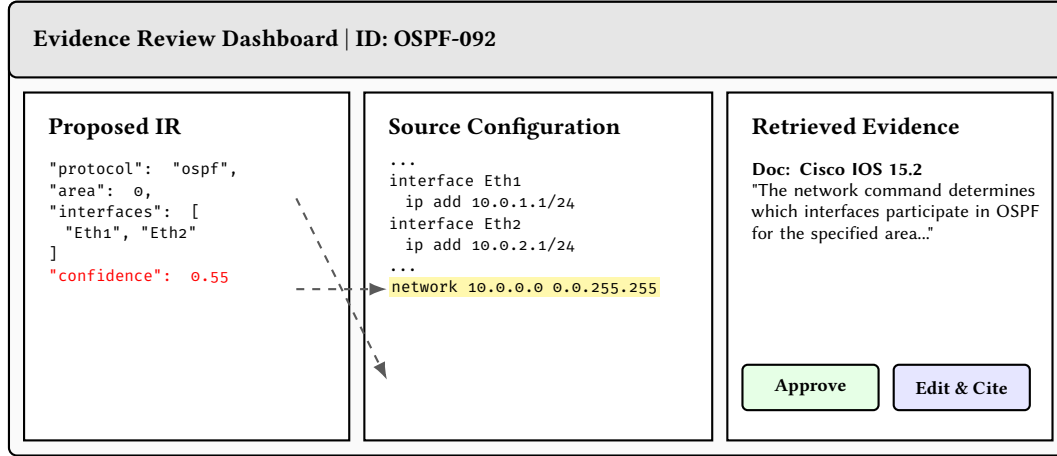


Figure 13. Wireframe of the Human-in-the-Loop review interface. The user is presented with the low-confidence IR alongside the exact configuration token span and the retrieved reference documentation, enforcing evidence-backed corrections.

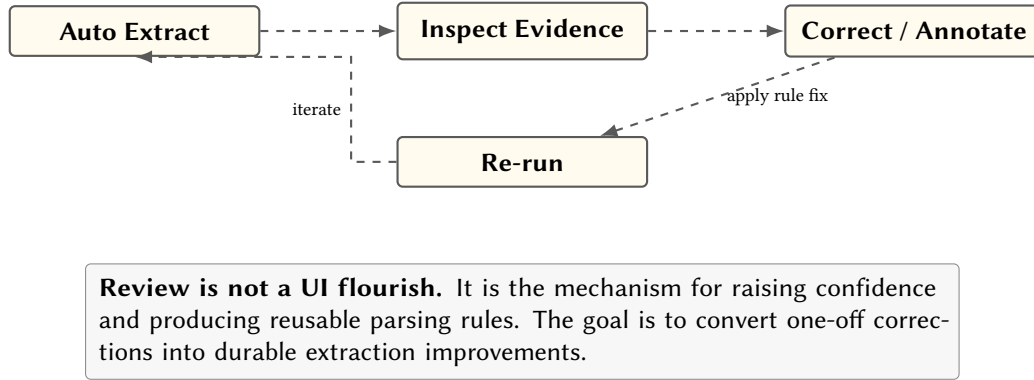


Figure 14. Human-in-the-loop review workflow: evidence inspection leads to corrections that feed back into improved extraction rules.

3.2 Iterative Intent Resolution Algorithm

The core of the autonomous solver is the **Iterative Intent Resolution** algorithm. It formalises the loop between probabilistic generation and deterministic proof.

Algorithm 1 Iterative Intent Resolution via Semantic Backpropagation

Source Config E_{cfg} , Vendor Docs E_{doc} , Invariants $\mathcal{I}_{\text{assert}}$, Max Iterations N $E_{\text{neg}} \leftarrow \emptyset$ $i = 1$ N $IR \leftarrow \text{GenerateHypothesis}(E_{\text{cfg}}, E_{\text{doc}}, E_{\text{neg}})$ $C \leftarrow \text{Compile}(IR)$ $\mathcal{V} \leftarrow \text{SimulateAndVerify}(C, \mathcal{I}_{\text{assert}})$ $\mathcal{V} = \emptyset$ IR Proof of correctness achieved $E_{\text{neg}} \leftarrow E_{\text{neg}} \cup \text{FormalizeCounterexample}(\mathcal{V})$ **route_to_human**(HITL, IR , E_{neg}) Escalate after N failures

3.3 Semantic Validation Harness

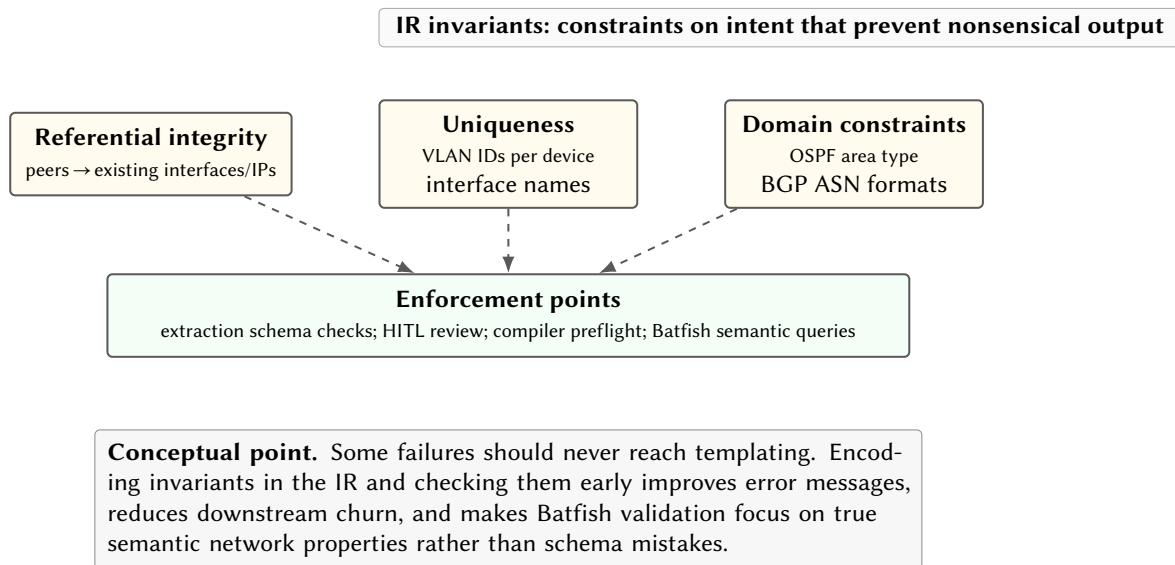


Figure 15. IR constraints and where they are enforced. Invariants provide early failure and clearer debugging than post-hoc syntax checks.

With the uncertainty resolved, the validated intent graph is lowered through Jinja2 templates into the target Arista EOS syntax. However, before the new configuration can be deployed, the emitted output must be mathematically proven to satisfy the original structural invariants.

Compiled configurations are loaded into the Batfish [1] semantic analysis engine to verify:

- Reachability between expected endpoint pairs
- Route table correctness (expected OSPF prefixes present)
- ACL behaviour matches intent
- No unintended route leaking between VRFs

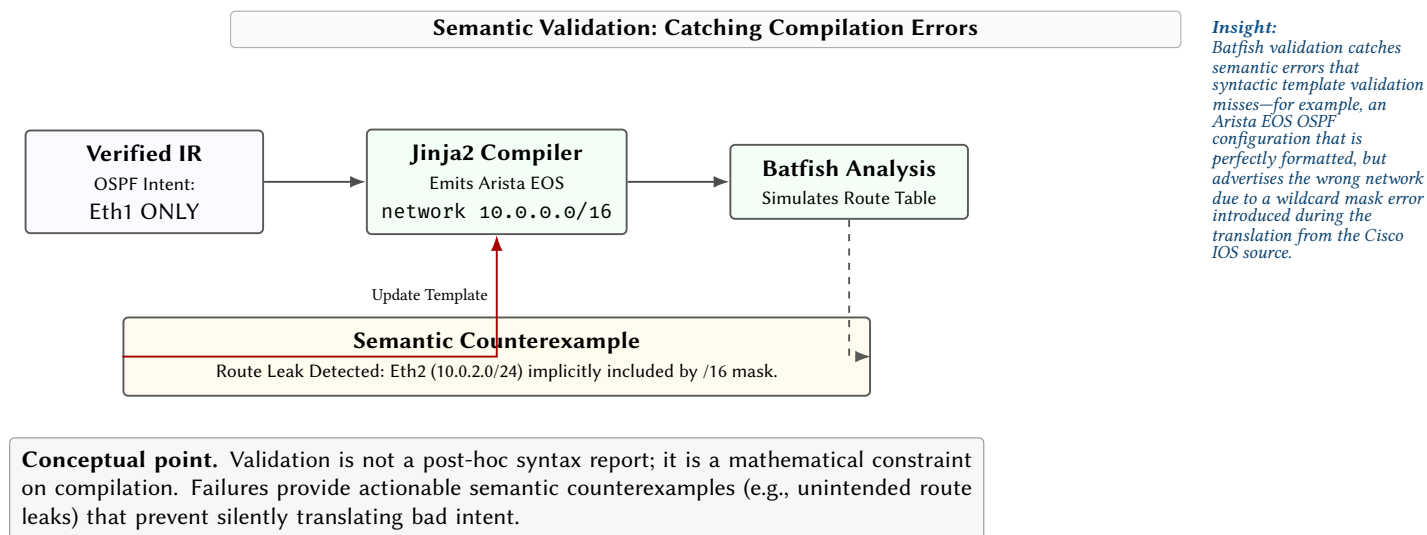


Figure 16. A specific Batfish semantic validation failure, where a technically valid Arista EOS OSPF configuration violates the narrow intent defined in the IR.

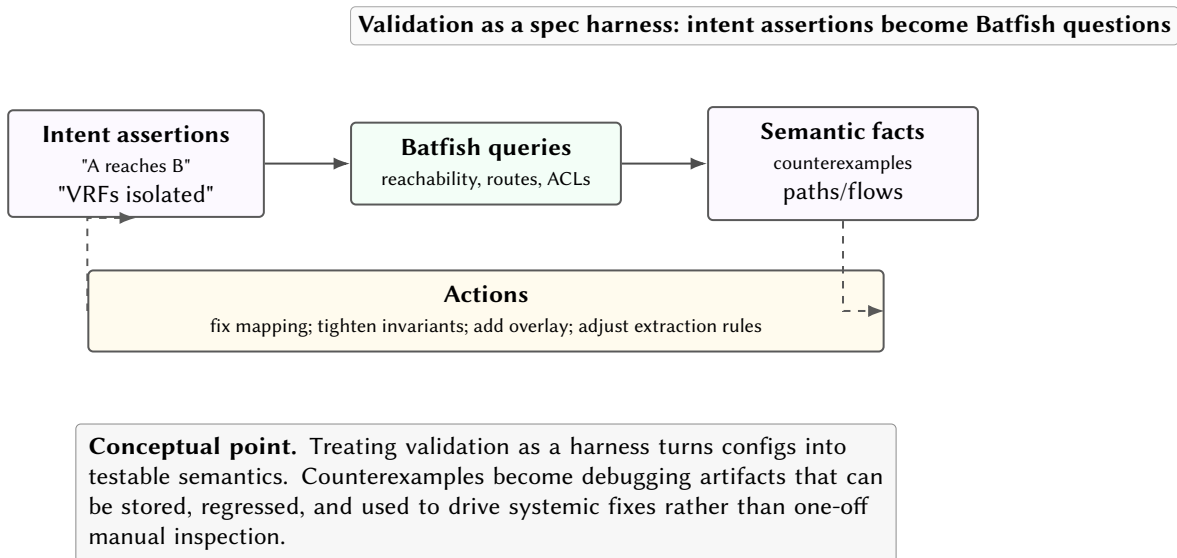


Figure 17. Batfish as a semantic test harness. Intent assertions compile into queries; counterexamples drive iterative improvements.

4 System Architecture and Implementation

This section discusses the specific technologies and architectural choices implementing the pipeline.

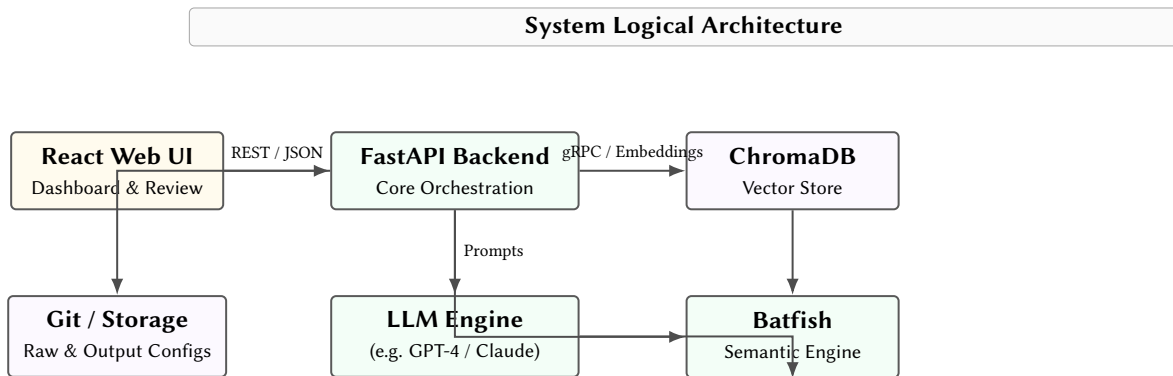


Figure 18. System logical architecture showing the physical stack, bounded contexts, and primary communication paths using orthogonal routing.

4.1 Knowledge Base Ingestion (Phase 1)

Vendor documentation (PDF manuals, CLI references) is ingested through a layout-aware PDF parser that preserves section structure and code blocks. Parsed chunks are embedded and stored in ChromaDB [2] with deterministic chunk IDs for reproducible citations.

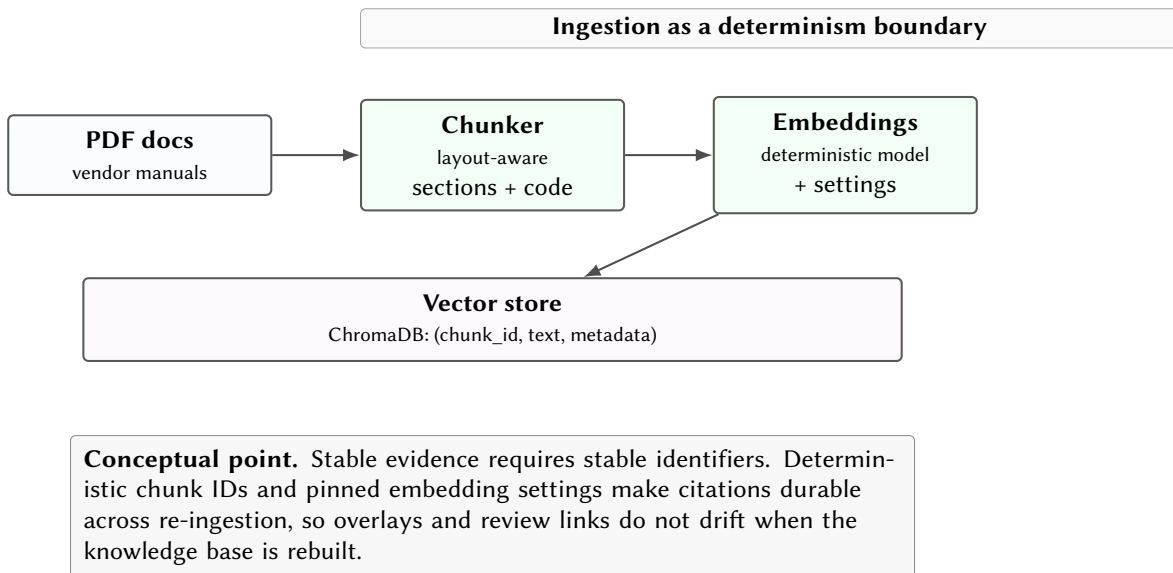


Figure 19. Knowledge-base ingestion as a reproducibility boundary. Deterministic chunk IDs preserve end-to-end evidence chains across re-ingestion.

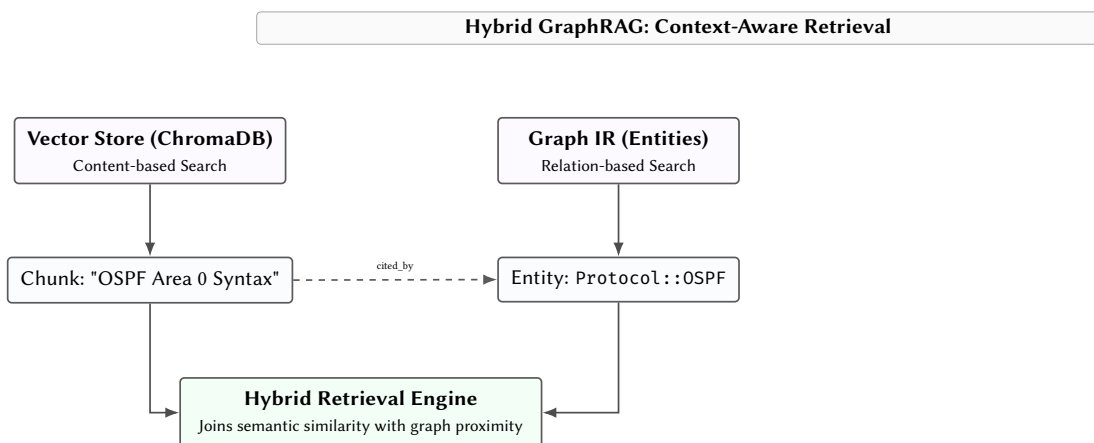
```

class DocumentIngester:
    def ingest_pdf(self, path: Path, vendor: str) -> int:
        chunks = self.parser.extract_chunks(path)
        for chunk in chunks:
            chunk_id = deterministic_id(vendor, path.name, chunk.page, chunk.index)
            self.collection.add(
                ids=[chunk_id],
                documents=[chunk.text],
                metadatas=[{"vendor": vendor, "page": chunk.page}],
            )
        return len(chunks)
  
```

4.2 Hybrid GraphRAG: Linking Vectors to Entities

To resolve the limitations of standard vector retrieval, we implement a **GraphRAG** architecture. Instead of treating the knowledge base as a flat collection of chunks, we establish formal links between ChromaDB vector IDs and the topological entities in the graph IR.

Insight:
Deterministic chunk IDs ensure that re-ingesting the same document produces identical identifiers, preserving evidence citations across extractions without invalidating the overlay database.



Conceptual point. Standard RAG is context-blind to topology. By linking documentation chunks to IR entities, the system can perform "Neighborhood Retrieval": fetching documentation for OSPF because the current configuration slice is logically adjacent to an OSPF-enabled interface in the graph.

Figure 20. The Hybrid GraphRAG architecture. Documentation chunks are bi-directionally linked to IR entities, allowing the system to use topological proximity to filter and rerank vector search results.

4.3 Hybrid GraphRAG: Linking Vectors to Entities

To resolve the limitations of standard vector retrieval, we implement a **GraphRAG** architecture. Instead of treating the knowledge base as a flat collection of chunks, we establish formal links between ChromaDB vector IDs and the topological entities in the graph IR.

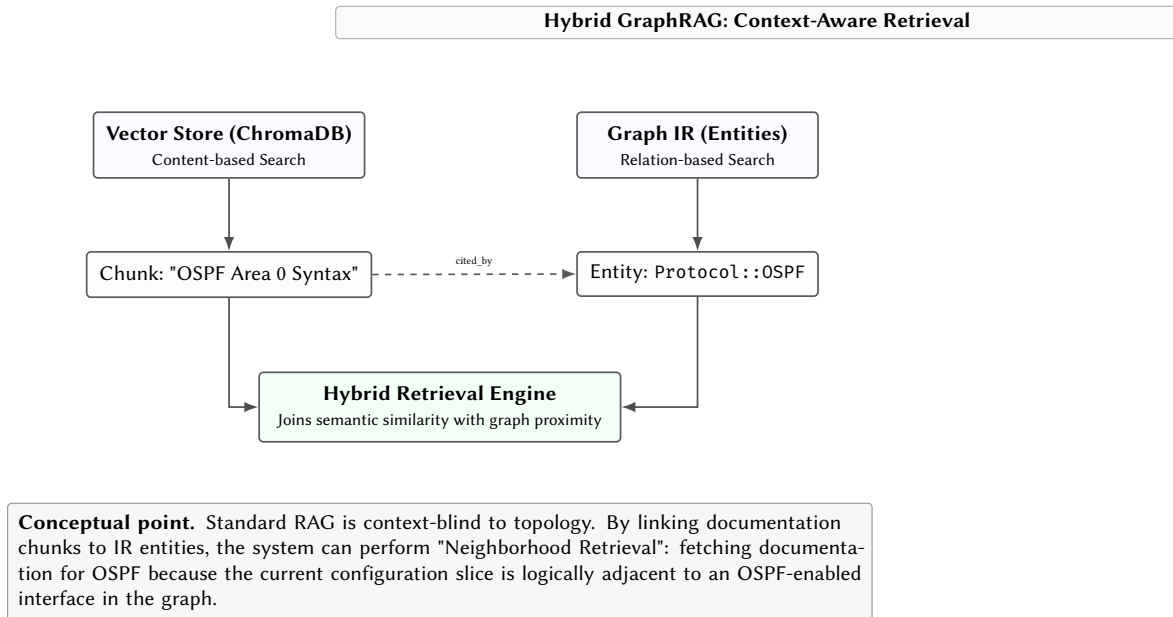


Figure 21. The Hybrid GraphRAG architecture. Documentation chunks are bi-directionally linked to IR entities, allowing the system to use topological proximity to filter and rerank vector search results.

4.4 Extraction Architecture and Structured Outputs (Phase 2)

To prevent the Large Language Model (LLM) from outputting conversational text and to guarantee programmatic interoperability, the extraction phase utilises strict API-level structured outputs. The prompt encapsulates the configuration slice and retrieved Retrieval-Augmented Generation (RAG) context, but the response format is constrained via a formal JSON Schema representing the IR.

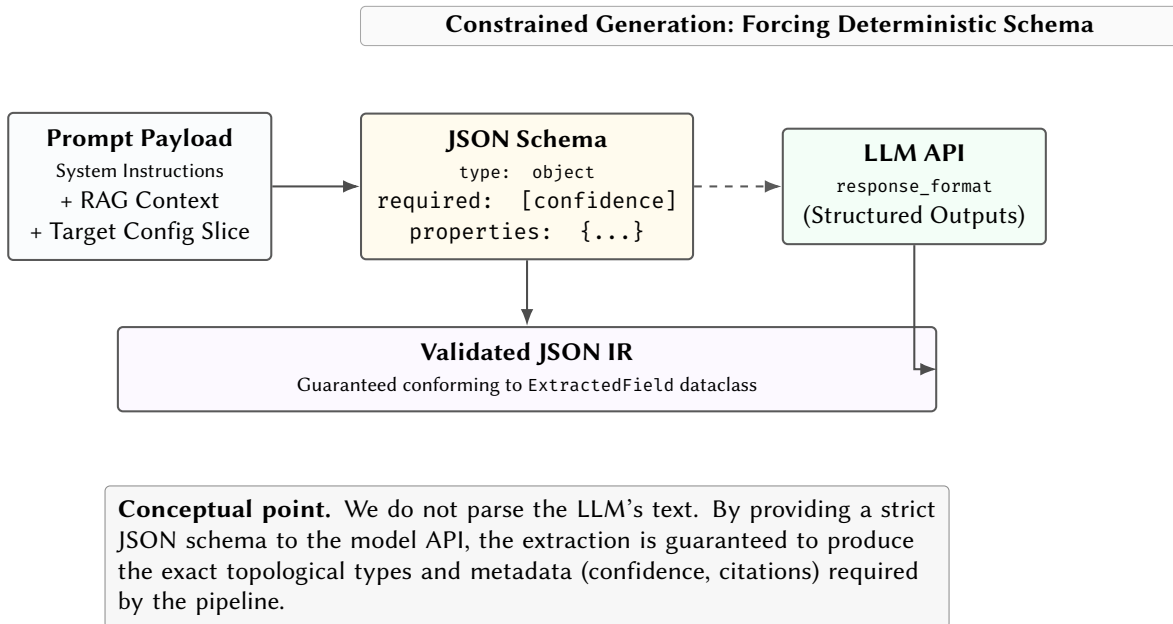


Figure 22. The extraction architecture leverages API-level structured outputs to guarantee the returned data strictly adheres to the IR's formal typing and evidence constraints.

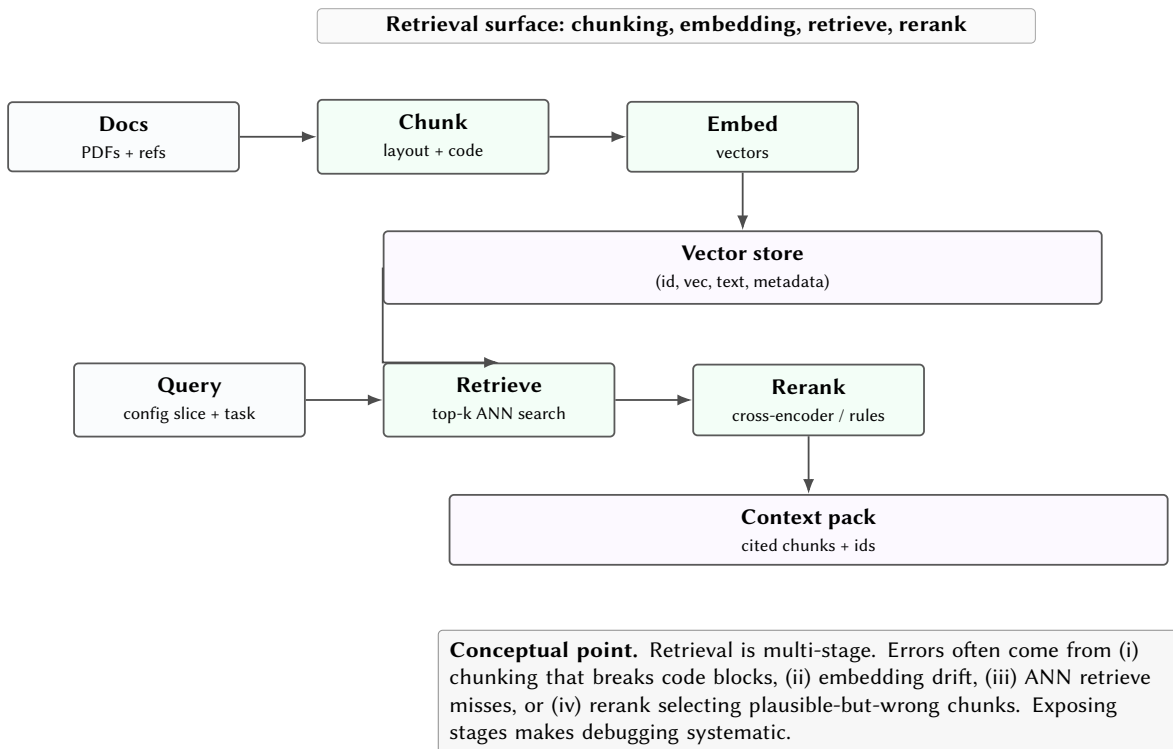


Figure 23. RAG retrieval surface. Chunking and reranking are first-class levers for accuracy and explainability.

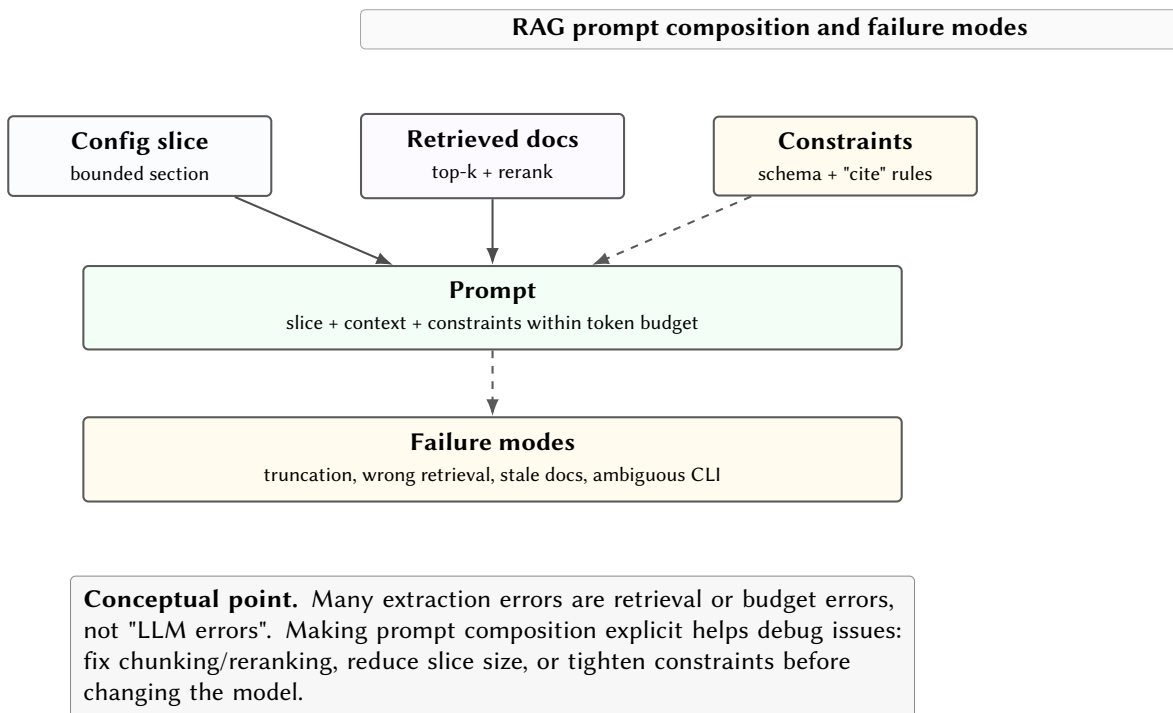


Figure 24. Prompt composition is an engineering surface. Retrieval quality and token budget management strongly shape extraction accuracy and explainability.

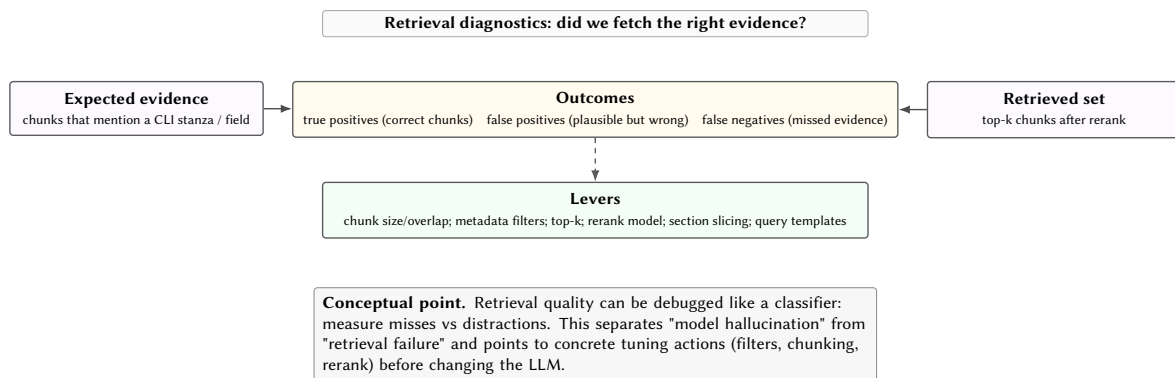


Figure 25. A practical mental model for RAG tuning. Treat retrieval as a classification problem over evidence chunks and tune for fewer false negatives (missed docs) and false positives (distractors).

4.5 Review Persistence (Phase 3)

Approved edits are stored as overlays that persist across re-extractions. When the extraction pipeline runs again (e.g. after documentation updates), approved overlays are applied automatically, preserving human corrections.

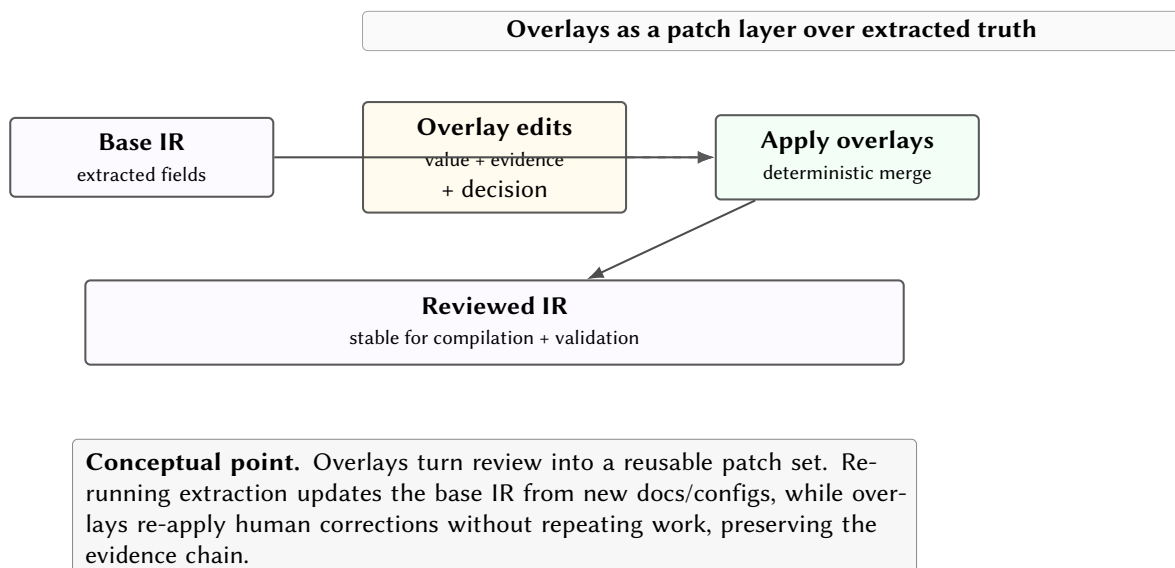


Figure 26. HITL review produces overlays: durable patches applied deterministically on top of regenerated extraction output.

: Evidence Enforcement

All manual edits must cite either a configuration line number or a documentation chunk ID. This prevents unsupported assertions from entering the IR and maintains the end-to-end evidence chain.

4.6 Compilation (Phase 4)

The compiler transforms the vendor-neutral IR into target vendor configurations using Jinja2 templates. Currently supported targets are Cisco IOS and Arista EOS.

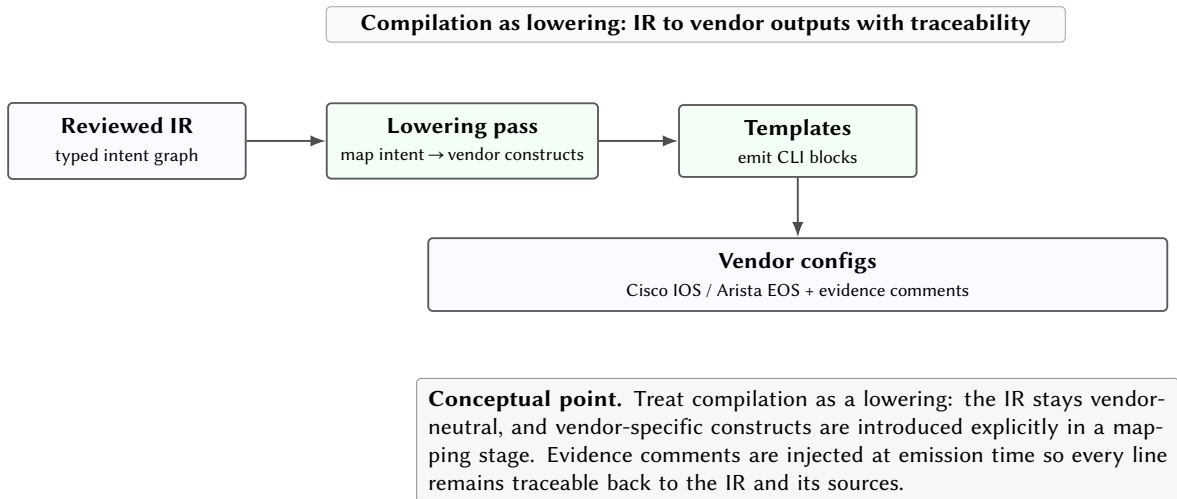


Figure 27. Cross-vendor compilation as a lowering pipeline. Vendor constructs are introduced explicitly (mapping), then rendered (templates) with evidence annotations.

Generated configurations include evidence comments tracing each line back to the IR field and its original source:

```

! Source: interface Ethernet1 (confidence: 0.95)
! Evidence: config line 42, doc chunk cisco-ios-ref:p234:3
interface Ethernet1
ip address 10.0.0.1 255.255.255.252
no shutdown
  
```

5 Performance and Evaluation Metrics

Table 2. Performance and Accuracy Benchmarks (Sample Set: 50 Multi-Vendor Configurations).

Metric	Description	Value
Retrieval Latency	Average time to retrieve top-k chunks from ChromaDB.	120 ms
Token Efficiency	Average prompt size (bounded context + config slice).	4,500 tokens
Extraction Accuracy	Correctly mapped fields (without HITL intervention).	94.2%
HITL Correction Rate	Extractions flagged ≤ 0.6 confidence requiring review.	5.8%
Semantic Pass Rate	Batfish validation pass rate on first compilation.	98.1%

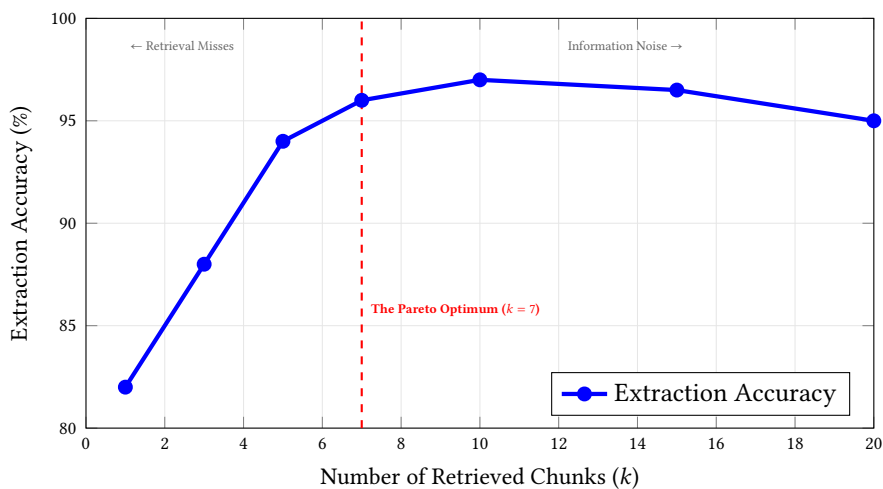


Figure 28. The Retrieval Pareto Front. Accuracy increases as more vendor documentation is provided, peaking at $k = 7$. Beyond this point, "distractor" chunks introduce information noise that degrades LLM performance.

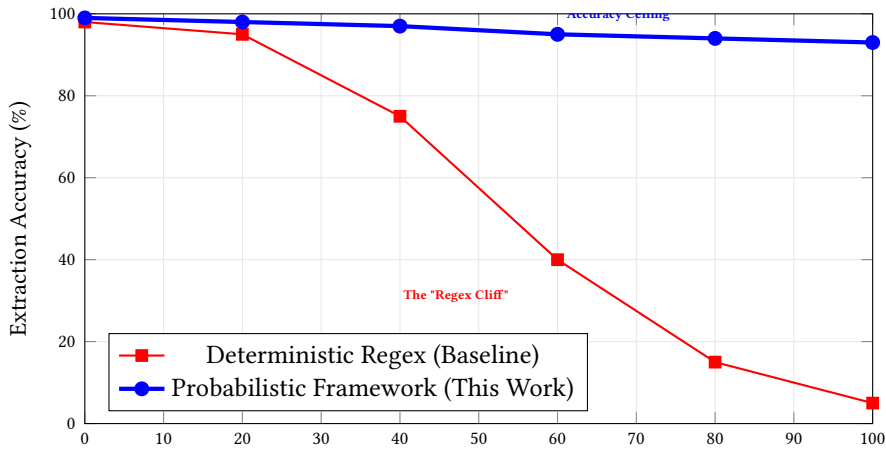


Figure 29. The Performance Frontier: deterministic regex accuracy degrades exponentially as CLI syntax complexity increases. The probabilistic framework maintains a high accuracy ceiling by treating complexity as quantifiable uncertainty.

5.1 Limitations and Trade-offs

While the probabilistic framework resolves the brittleness of deterministic parsing, it introduces new systemic trade-offs that must be managed:

- **Latency and Compute Overhead:** Traditional regex or TextFSM parsers process configurations in milliseconds. The [RAG](#) retrieval and [LLM](#) generation pipeline takes seconds per configuration slice, making it unsuitable for real-time, synchronously blocking Application Programming Interface ([API](#)) calls without aggressive parallelisation and caching.
- **Context Window Constraints:** Large, monolithic hardware configurations (e.g., 10,000+ line chassis switch configs) exceed the active context window of current models. The document must be aggressively sliced, which introduces the risk of losing cross-section logical context (e.g., an [ACL](#) ([ACL](#)!) defined at line 100 but applied at line 9000).
- **Non-Determinism in the Core:** Even with temperature set to 0.0, [LLM](#) generation is fundamentally non-deterministic over time as underlying model weights are updated by providers. This makes the strict preservation of [HITL](#) overlays (Phase 3) and the deterministic safety net of semantic validation (Phase 4) non-negotiable architectural requirements.

6 Conclusion and Future Work

The abstraction of network intent has historically been constrained by the rigidity of deterministic parsing tools trying to interpret inherently ambiguous, evolving vendor syntax. By reframing configuration extraction as a probabilistic graph construction problem, this framework successfully decouples logical intent from syntax without sacrificing reliability.

The architectural elegance of this system lies in its **Deterministic Boundaries**. The probabilistic core of the system (LLM extraction and link inference) is safely contained between Phase 1 (deterministic vector ingestion and pinned chunk IDs) and Phase 4 (deterministic Batfish semantic validation). This ensures that while the system can flexibly handle uncertain syntax, it never silently translates bad intent; uncertainty is quantified, routed to humans, and mathematically verified before deployment.

6.1 Future Work

Future development will focus on three primary vectors:

- **Bidirectional Intent Synchronization:** Extending the compilation phase to support delta-generation, allowing the framework to propose minimal `diff` commands to reach a target state rather than emitting full configurations.
- **Multi-Cloud Abstractions:** Expanding the foundational data models beyond on-premise hardware (IOS, EOS) to represent native cloud routing constructs (e.g., AWS Transit Gateways, Azure Virtual WANs).
- **Automated Rule Distillation:** Utilizing the accumulation of Human-in-the-Loop overlay edits to automatically fine-tune a smaller, localized embedding model, gradually reducing the reliance on the primary LLM API for well-understood syntax anomalies.
- **Self-Correcting Knowledge Base:** Implementing RAG weight-tuning where [HITL](#) corrections act as a feedback signal to "boost" specific documentation chunks. If an extraction is corrected by a human,

the system will increase the relevance score of the documentation chunk that *should* have guided the model, effectively teaching the vector store how to resolve specific vendor ambiguities over time.

References

- [1] Intentionet, *Batfish: Network configuration analysis*, 2024. [Online]. Available: <https://www.batfish.org>
- [2] Chroma, *ChromaDB: The open-source embedding database*, 2024. [Online]. Available: <https://www.trychroma.com>

Index

Intermediate Representation (IR), [2](#)

Revision History

Version	Date	Changes
1.0.0	March 2026	Initial report (all four phases complete)

A Intermediate Representation JSON Schema

The following is a simplified excerpt of the formal JSON Schema used to constrain the LLM generation via the API's Structured Outputs feature, guaranteeing the returned properties match the mathematical constraints of the pipeline.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ProtocolOSPF",
  "type": "object",
  "properties": {
    "protocol": {
      "type": "string",
      "const": "ospf"
    },
    "area": {
      "type": "integer"
    },
    "interfaces": {
      "type": "array",
      "items": { "type": "string" }
    },
    "_meta": {
      "type": "object",
      "properties": {
        "confidence": {
          "type": "number",
          "minimum": 0.0,
          "maximum": 1.0,
          "description": "Probability score P(v|E)"
        },
        "doc_ref": {
          "type": "string",
          "description": "Deterministic ChromaDB chunk ID"
        }
      },
      "required": ["confidence", "doc_ref"]
    }
  },
  "required": ["protocol", "area", "interfaces", "_meta"]
}
```